Universal External Ballistics Calculator

Technical documentation for ballistics.py

Point-mass trajectory solver with G1 / G7 / custom drag, atmosphere, wind, inclined fire, Coriolis effect, spin drift and zeroing

Version	2.0 (metric edition)
Language	Python 3.8+ (standard library only; matplotlib optional for plots)
Model	3-DOF point mass, RK4 integration
Scope	External ballistics (flight after launch)
Units	Metric only: m, m/s, g, mm, cm, C, hPa, J. SI internally. Ballistic coefficient stays in lb/in2, the published unit.
Ways to use	Answer questions (interactive), one command, or import as a Python module

1. Introduction and scope

ballistics.py is a general-purpose external ballistics calculator. Given a projectile, its launch speed and the surrounding conditions, it predicts where the projectile is, how fast it is moving and how much energy it carries at every distance down-range. The same engine serves sporting rifle bullets, air-gun pellets, arrows and ordinary balls, because it only needs a description of the projectile's aerodynamic drag.

It is meant for target shooting, range estimation, education and physics coursework. It prints the familiar range table: speed, Mach number, energy, time of flight, drop, wind drift and scope corrections in MOA and milliradians (MIL). Everything is metric: ranges in metres, speeds in m/s, drop and drift in centimetres, energy in joules.

Out of scope. The program models flight only. It does not model anything inside a launcher, terminal effects, or the design of any device. It is a prediction tool for flight paths, comparable to open-source sporting ballistic calculators.

Main capabilities

Feature	Description
Drag models	G1 or G7 reference curves scaled by a ballistic coefficient (BC); a user-supplied Mach/Cd CSV curve; or a direct Cd with mass and diameter.
Atmosphere	ICAO standard atmosphere by altitude, or measured temperature (C), station pressure (hPa) and relative humidity.
Wind	Constant horizontal wind (m/s) from any direction relative to the line of fire.
Inclined fire	Uphill/downhill shots in a slope-aligned frame with tilted gravity.
Coriolis	Full vector Coriolis acceleration from latitude and azimuth.
Spin drift	Miller stability factor and the Litz spin-drift approximation.
Zeroing	Solves the bore angle so the path crosses the line of sight at any zero range and sight height.
Friendly use	Run with no options and it asks questions, prints a plain-English summary and shows the equivalent command.
Outputs	Console table, CSV export, PNG plot, importable Python API.
Verification	Built-in --selftest (analytic vacuum comparison, convergence, sanity checks).

2. Quick start

No installation is needed beyond Python 3. Save ballistics.py anywhere and run:

```
python ballistics.py --selftest
python ballistics.py --mv 823 --bc 0.462 --model G1 --weight 10.9 --diameter 7.82 \
    --sight-height 38 --zero 100 --max-range 1000 --step 100 \
    --wind-speed 5 --wind-dir 90 --lat 40 --azimuth 0 --length 30.9 --twist 280
```

All inputs are metric: m/s, grams, millimetres, metres, Celsius, hPa and m/s wind. The ballistic coefficient is always entered in lb/in2 because that is how manufacturers publish it.

Easiest way: answer questions

Run the program with no options. Press Enter to accept the value in [brackets]. At the end it prints the results, a short summary, and the one-line command that repeats the same calculation.

```
Ballistics Calculator (metric)
Press Enter to accept the value in [brackets].

What are you launching?
  1 = a bullet or pellet with a ballistic coefficient (BC)
  2 = something with a plain drag coefficient (Cd)
  3 = a ready-made ball (baseball, golf_ball, tennis_ball)
  Choose 1, 2 or 3 [1]: 1
  Launch speed (m/s) [823]: 823
  Weight (g) [10.9]: 10.9
  Diameter (mm) [7.82]: 7.82
```

```
  Drag model (G1 or G7) [G1]: G7
  Ballistic coefficient (lb/in^2) [0.462]: 0.243
  Sight height above barrel (mm) [38]: 38
  Zero range (m, 0 = none) [100]: 100
  Table goes out to (m) [1000]: 600
  Table step (m) [100]: 100
  Wind speed (m/s, 0 = calm) [0.0]: 3.5
  Wind comes FROM (0 ahead, 90 right, 180 behind, 270 left) deg [90.0]: 90
  Standard weather (sea level, 15 C)? yes/no [yes]: yes
  Include Earth's rotation (Coriolis)? yes/no [no]: no
```

Using it as a library

```
from ballistics import *
bullet = from_bc('G7', 0.243, 0.0113, 0.00782, length=0.0315, twist=0.254)
env = Env(Air.standard(1500), wind=4.0, wind_from=90, lat=45, azimuth=270)
sol = compute(bullet, env, 800.0, sight=0.04, zero=100.0, max_range=1200, step=100)
for r in sol.rows:
    print(r.range_m, r.speed, r.drop, r.wind_drift)
```

Lengths are metres, masses kilograms, speeds m/s. Non-firearm example (a thrown baseball):

```
python ballistics.py --preset baseball --mv 40 --angle 30 --max-range 100 --step 10
```

Command-line reference

Option	Meaning
--mv	Launch velocity, m/s.
--model / --bc	G1, G7 or CUSTOM reference curve and BC (lb/in2).
--cd / --drag-csv	Direct Cd, or a CSV of mach,cd rows (non-numeric lines ignored).
--weight / --diameter	Mass in grams and diameter in mm.
--length / --twist	Projectile length and barrel twist (mm per turn) for spin drift; --left-twist for left-hand rifling.
--sight-height / --zero	Sight axis above bore (mm) and zero range (m).
--angle	Manual bore elevation above the line of sight in degrees (skips zeroing).
--max-range / --step	Table extent and spacing, in metres.
--temp / --pressure / --humidity / --altitude	Air state: C, station pressure in hPa, percent, and metres. Otherwise ICAO by altitude.
--wind-speed / --wind-dir	Wind in m/s and the direction it comes from: 0 head, 90 from the right, 180 tail, 270 from the left.
--incline	Shot angle in degrees (+ uphill).
--lat / --azimuth	Latitude (enables Coriolis) and fire direction clockwise from true north.
--dt	Integration step in seconds (default 0.0005).
--csv / --plot	Export the table / save a trajectory PNG.
--interactive	Ask questions instead (also the default with no options).
--selftest	Run the built-in checks.

3. Physical model

3.1 Frame and equations of motion

The solver works in a right-handed frame aligned with the line of sight: x is down-range, y is up, z is to the shooter's right. The projectile is a point mass with position (x, y, z) and velocity v. Newton's second law gives

$$dv/dt = g - k(M)\,\rho_a\,|v_r|\,v_r - 2\,\Omega \times v$$

where g is gravity (tilted for sloped shots), $v_r = v - w$ is velocity relative to the air (wind w), ρ_a is air density, Omega is Earth's angular velocity and k(M) is the drag factor below.

3.2 Drag and the ballistic coefficient

Drag deceleration is $a_{drag} = (1/2)\ rho_a\ v_r^2\ Cd(M)\ A / m$, with Mach number $M = v_r/c$. In direct-Cd mode the program uses this with the supplied Cd and $A = pi\ d^2/4$. Real bullet Cd curves are rarely known, so a bullet is compared with a reference shape: $BC = SD / i$, where $SD = m / d^2$ is the sectional density and i the form factor. The deceleration becomes $(1/2)\ rho_a\ v_r^2\ Cd_{ref}(M)\ (pi/4) / BC_{SI}$, with BC_{SI} = BC x 703.07 kg/m^2 for a BC quoted in lb/in^2. The code folds this into one factor k(M), so all drag modes share one integrator.

3.3 Reference drag curves

G1 is a flat-based blunt-ogive shape, historically the most common reference. G7 is a long, boat-tailed, pointed shape that resembles modern long-range bullets. A real bullet's effective G1 BC drifts with speed, so a single G7 BC is usually more trustworthy through the transonic region. You can also supply your own Mach/Cd table with --drag-csv.

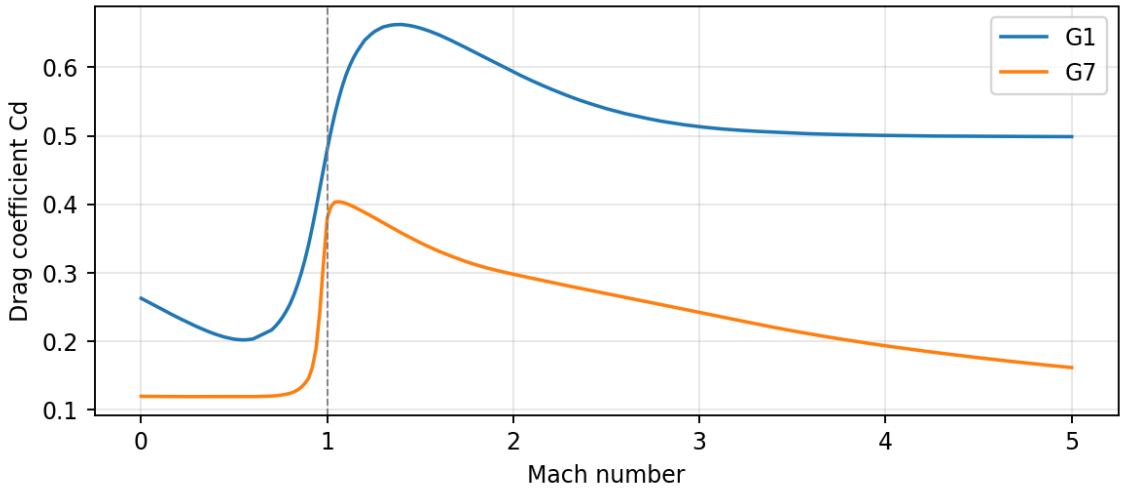


Figure 1. Built-in G1 and G7 drag coefficient curves. The sharp rise near Mach 1 is the transonic drag rise.

Data note: the tables were entered from widely published standard drag functions. For certified numbers, compare against your preferred authoritative source or replace them via CSV.

3.4 Atmosphere

Air density comes from the ideal gas law for a dry-air/water-vapour mixture: $rho_a = p_d/(R_d\ T) + p_v/(R_v\ T)$, with $p_d = p - p_v$, T in kelvin, R_d = 287.058 and R_v = 461.495 J/(kg K). Vapour pressure is $p_v = RH\ x\ p_{sat}$, with saturation pressure from the Buck (1981) formula. Speed of sound is $c = sqrt(1.4\ R_d\ T)$. Without weather input the ICAO standard atmosphere is used: $T = 288.15 - 0.0065\ h$ and $p = 101325\ (T/288.15)^{5.25588}$, giving 1.225 kg/m^3 at sea level. Density and speed of sound are held constant along the flight, which suits normal sporting ranges but not very steep or very long flights.

3.5 Wind

Wind is a constant horizontal vector. For wind coming from angle theta (0 = head, 90 = from the right) its components are (-W cos theta, 0, -W sin theta), rotated into the slope frame. Because drag depends on velocity relative to the air, crosswind deflection comes out of the integrator naturally; no separate wind-drift formula is used.

3.6 Inclined fire

For slope angle alpha the frame is tilted so x follows the line of sight and gravity becomes (-g sin alpha, -g cos alpha, 0). The bore angle found at the zero range on flat ground is reused, as in real shooting where the sight stays fixed. Both uphill and downhill shots then hit high relative to a flat-fire hold; the 'cosine rule' emerges from the physics and the code adds drag on top.

3.7 Coriolis effect

Earth's rotation vector in the line-of-fire frame for latitude phi and azimuth A is Omega = Omega_E (cos phi cos A, sin phi, -cos phi sin A), and -2 Omega x v is added to the acceleration. This gives both the horizontal deflection (right in the northern hemisphere) and the vertical (Eotvos) component that depends on firing direction. Omega_E = 7.2921159e-5 rad/s.

3.8 Spin stability and spin drift

A spin-stabilised bullet drifts sideways in the direction of twist. The code estimates the Miller stability factor $S_g = 30\ m / (t^2\ d^3\ l\ (1 + l^2)) \times (v/2800)^{1/3} \times ((TF+460)/519) \times (29.92/p_{inHg})$, with m in grains, d in inches, and twist t and length l in calibres (the code converts from metric internally). If S_g is at least 1.0, the Litz approximation applies: drift (in) = $1.25\ (S_g + 1.2)\ TOF^{1.83}$, converted to metres, with the sign set by the rifling twist. Without length and twist, spin drift is omitted. It is an empirical fit, not a first-principles model.

3.9 Zeroing

The sight line is straight, at height h above the bore axis. For zero range Z the program bisects on the bore elevation angle until the simulated height at x = Z equals h (about 45 halvings). The zero is always solved for still air, flat ground and a non-rotating Earth in the current atmosphere, because a zero belongs to the rifle and scope, not to the day's wind. Corrections are reported as -atan(drop/x) for elevation and -atan(lateral/x) for windage, in MOA (3437.75 per radian) and MIL (1000 per radian).

3.10 Numerical integration

Classical fourth-order Runge-Kutta (RK4) with a fixed 0.5 ms step by default. Positions advance with the trapezoid rule on the updated velocities, and results at requested ranges come from linear interpolation between the two nearest steps. Section 8 shows halving the step changes the 1000 m drop by a fraction of a millimetre.

4. Software architecture

One file, small pieces. Dataclasses hold inputs; functions do the numerics.

Element	Role
Drag	Piecewise-linear Cd(Mach) with flat extrapolation; constructors for constant Cd and CSV.
G1, G7	Standard reference drag tables (lists of Mach, Cd).
Air	Temperature, pressure, humidity to density and speed of sound; standard() constructor; density override (used for the vacuum test).
Bullet	Mass, diameter, drag curve, optional BC, length and twist. drag_factor(mach) gives the unified drag factor. Built with from_bc() or from_cd().
Env	Air, wind speed and direction, incline, latitude, azimuth, gravity.
fly()	RK4 integrator in the slope frame; returns the sampled track.
at_range(), solve_zero()	Interpolate the track at a range; bisect the bore angle for a zero.
stability(), spin_drift()	Miller stability factor and lateral spin drift.
compute()	High-level entry point: zeroing, integration, interpolation, range table and summary (peak height, transonic and subsonic range).
report(), write_csv(), plot()	Presentation layer.
selftest()	Verification suite.
wizard(), parser(), build(), run(), main()	Questions, command-line options, and turning options into a Bullet and Env.

Design choices

Unified drag factor. BC-based and Cd-based projectiles both reduce to k(M) in m^2/kg, so the integrator never branches on drag model. **SI internally.** The command line is metric, so conversion is just grams to kg and mm to m. **No dependencies.** Pure Python; matplotlib is imported only when a plot is requested. **No global state.** Trajectories are reproducible and easy to unit-test. A 1000 m solution takes a fraction of a second on an ordinary laptop.

5. Code walkthrough in plain language

For readers who want to understand or tweak the code. The ideas are ordinary physics: things fall, air pushes back, and we predict the path by moving the projectile a tiny bit at a time.

5.1 The big picture

Step	What happens	Where
1	Read your answers or options and convert grams and millimetres to kg and m.	main(), wizard(), build(), run()
2	Describe the projectile: mass, size, how much the air slows it.	Bullet
3	Describe the surroundings: air, wind, slope, latitude.	Air, Env
4	Find the barrel angle that zeros the sights at your chosen distance.	solve_zero()
5	Fly the projectile in thousandths of a second, recording where it is.	fly()
6	Read the flight at 100, 200, 300 ... m and build the table rows.	compute(), at_range()
7	Print the table and a plain-English summary; optionally save CSV or a plot.	report(), write_csv(), plot()

5.2 How the file is laid out

Top to bottom: constants; G1 and G7 tables; Drag; Air; Bullet with from_bc() and from_cd(); Env and the frame helpers tilt() and earth_spin(); fly(), at_range() and solve_zero(); stability() and spin_drift(); the Row and Solution containers with compute(); PRESETS (baseball, golf_ball, tennis_ball); selftest(); and finally the user-facing functions.

5.3 Ideas used everywhere

Metric inside, metric outside. Everything is metres, kilograms and seconds internally; the only conversions are g to kg, mm to m and the display units (cm, J). **Dataclasses are labelled boxes.** Air, Bullet, Env, Row and Solution hold related values under names, so you write row.speed instead of remembering an index. **One drag number for everything.** Bullet.drag_factor(mach) returns how strongly the air slows the projectile at that speed, whichever way it was described.

```
Drag(G1)(1.0)                # drag coefficient at Mach 1 -> 0.4805
Air.standard().density()     # air density on a standard day -> 1.225 kg/m3
Air.standard().sound()       # -> about 340.3 m/s
bullet = from_bc('G1', 0.462, 0.0109, 0.00782)
bullet.drag_factor(2.0)      # drag strength at Mach 2
```

5.4 The heart of it: a toy flight loop

The idea fits in a dozen lines: work out the forces, change the velocity a little, change the position a little, repeat until it lands. This toy uses a simple drag rule and no wind or weather:

```
import math
g = 9.81
dt = 0.001
k = 0.0004
x, y = 0.0, 0.0
angle = math.radians(30)
vx, vy = 100 * math.cos(angle), 100 * math.sin(angle)

while y >= 0:
```

```
    speed = math.hypot(vx, vy)
    ax = -k * speed * vx
    ay = -g - k * speed * vy
    vx += ax * dt
    vy += ay * dt
    x += vx * dt
    y += vy * dt
print(f"It landed {x:.0f} m away")
```

It prints 713 m (the same throw without drag would travel about 883 m). The real fly() is the same loop with four upgrades:

Toy version	Real version
One drag constant k	Drag looked up from the G1/G7/custom curve at the current Mach number and scaled by air density and BC.
Forward and up (2-D)	Forward, up and sideways (3-D), so wind and Coriolis can push sideways.
Euler update (one guess per step)	Runge-Kutta 4: four guesses per step, blended. Far more accurate for the same step.
Still air	Drag uses speed relative to the air, so wind is subtracted first.
Runs until it lands	Runs until it reaches the requested distance.

The physics lives in one small function inside fly():

```
def acc(vx, vy, vz):
    rx, ry, rz = vx - wx, vy - wy, vz - wz
    s = math.sqrt(rx * rx + ry * ry + rz * rz)
    k = b.drag_factor(s / c) * rho * s
    return (g[0] - k * rx - 2 * (sy * vz - sz * vy),
            g[1] - k * ry - 2 * (sz * vx - sx * vz),
            g[2] - k * rz - 2 * (sx * vy - sy * vx))
```

5.5 How the sights get zeroed

There is no formula. The code plays guess and halve: fire at a low angle and a high angle; if the middle angle lands low, aim higher and drop the lower half; otherwise drop the upper half. After 45 halvings the answer is far more precise than needed.

```
lo, hi = -0.05, 0.35
for _ in range(45):
    mid = 0.5 * (lo + hi)
    if miss(mid) < 0:
        lo = mid
    else:
        hi = mid
angle = 0.5 * (lo + hi)
```

5.6 Reading the output table

Column	Meaning
Range	Distance down-range, m.
Speed / Mach	Speed in m/s and as a multiple of the speed of sound.
Energy	Kinetic energy, 1/2 m v^2, in joules.
Time	Seconds since launch.
Drop	Height versus the line of sight, cm. Minus is below. For the ball presets it is height above the launch point.
Up (MOA, MIL)	How far to dial up (or hold over) to hit the line-of-sight point. Positive = up.
Drift	Sideways offset from wind, Coriolis and spin drift, cm. Plus = right.
Wind (MOA, MIL)	Windage correction. Plus = dial right, minus = dial left. A wind from the right gives a plus number.

One MOA is about 2.9 cm per 100 m; one MIL is 10 cm per 100 m.

5.7 Sign conventions

Quantity	Positive means	Negative means
Height / drop	Above the line of sight	Below the line of sight
Sideways offset (Drift)	Right	Left
Incline angle	Shooting uphill	Shooting downhill
Wind direction (comes FROM)	0 = ahead, 90 = your right, 180 = behind, 270 = your left	-
Latitude	Northern hemisphere	Southern hemisphere
Azimuth	Degrees clockwise from true north (90 = east)	-

5.8 Common jobs

I want to...	Do this
Use my own drag curve	Make a text file with lines like 0.5,0.21 (Mach, Cd) and run with --drag-csv mycurve.csv --bc 1.0 (BC 1.0 means exactly this curve). For a direct Cd, omit --bc and give --weight and --diameter.
Save results for a spreadsheet	Add --csv results.csv (values are in SI units).
Get a picture	Add --plot path.png (needs pip install matplotlib).
Change table spacing	--max-range 1500 --step 50.
Add a new ball	Add a line to PRESETS: 'name': (mass_kg, diameter_m, Cd, typical_speed_m_s).
More accuracy or speed	Change --dt. The 0.0005 s default is already very precise (Section 8).
Check the program	python ballistics.py --selftest
Use it in my own program	from ballistics import *, build a Bullet and Env, and call compute().

6. Worked example

A 10.9 g, 7.82 mm bullet with a G1 BC of 0.462, launched at 823 m/s, zeroed at 100 m with a 38 mm sight height, in ICAO standard air, with a 5 m/s full-value crosswind from the right, at 40 degrees north latitude firing due north, bullet length 30.9 mm, 280 mm right-hand twist. These are illustrative sporting-ammunition-class numbers, not the specification of any product.

Range (m)	Speed (m/s)	Mach	Energy (J)	TOF (s)	Drop (cm)	Up (MOA)	Up (MIL)	Drift (cm)	Wind (MOA)	Wind (MIL)
0	823	2.42	3691	0.000	-3.8	0.00	0.00	0.0	0.00	0.00
100	758	2.23	3131	0.127	-0.0	0.00	0.00	-2.3	0.78	0.23
200	696	2.05	2640	0.264	-13.3	2.29	0.67	-9.5	1.63	0.48
300	637	1.87	2211	0.415	-46.9	5.38	1.56	-22.4	2.57	0.75
400	581	1.71	1838	0.579	-104.8	9.01	2.62	-41.7	3.58	1.04
500	528	1.55	1519	0.760	-191.9	13.19	3.84	-68.2	4.69	1.36
600	479	1.41	1250	0.958	-314.2	18.01	5.24	-102.8	5.89	1.71
700	434	1.28	1028	1.178	-479.5	23.55	6.85	-146.4	7.19	2.09
800	395	1.16	850	1.420	-696.9	29.95	8.71	-199.6	8.58	2.49
900	362	1.06	713	1.684	-977.3	37.33	10.86	-262.7	10.03	2.92
1000	336	0.99	614	1.972	-1332.7	45.81	13.33	-335.0	11.52	3.35

Zero bore angle: 0.0656 deg. Miller stability S_g = 2.03. Maximum ordinate 0.5 cm at 74 m. The bullet drops below Mach 1.2 near 764 m and below Mach 1 near 980 m, where drag and stability sensitivity rise sharply. 'Up' is the elevation correction (positive = dial up); 'Wind' is the windage correction (positive = dial right, so a wind from the right shows a positive number to steer into it). The Drift column is negative because that wind pushes the bullet left.

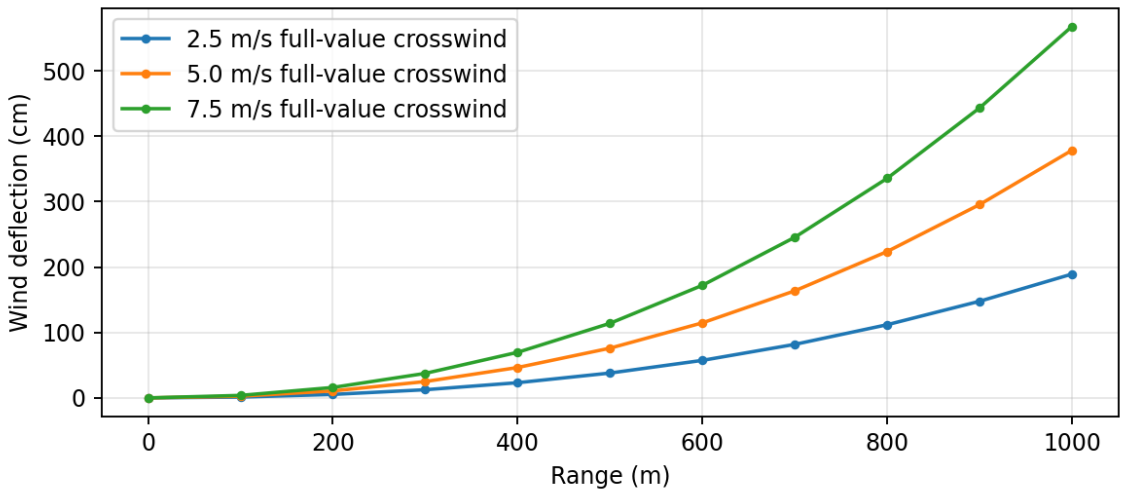


Figure 2. Wind deflection versus range for 2.5, 5 and 7.5 m/s full-value crosswinds. Deflection grows faster than linearly because the bullet slows down (lag time).

Atmospheric sensitivity

Density is the most important environmental variable for long-range flight. Figure 3 repeats the zeroed shot in four atmospheres and Table 2 lists the resulting drop. Thin, hot air reduces drag and drop; cold, dense air does the opposite.

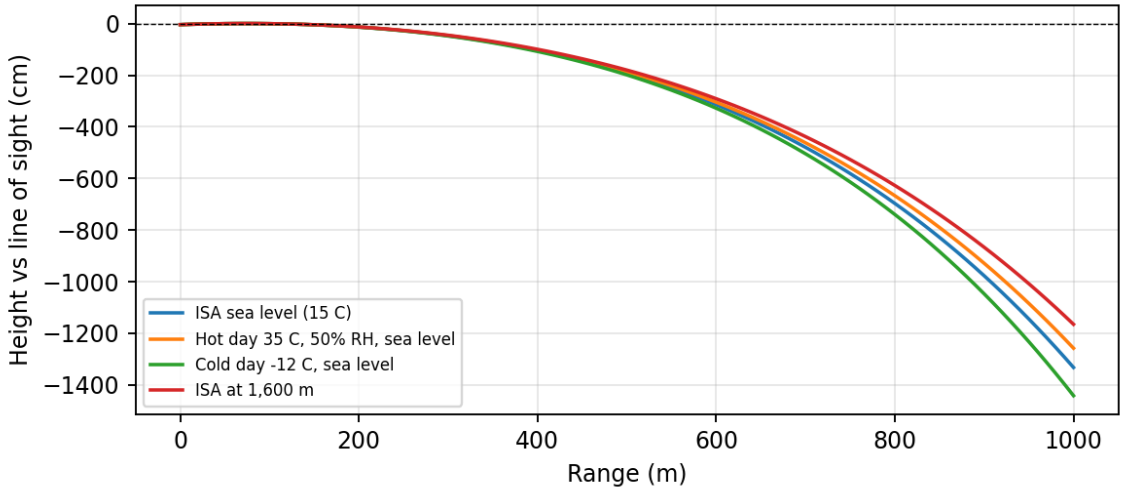


Figure 3. Trajectory with the rifle zeroed at 100 m in the same air it is fired in, under four atmospheres.

Atmosphere	Drop at 500 m (cm)	Drop at 1000 m (cm)
ISA sea level (15 C)	-191.9	-1332.7
Hot day 35 C, 50% RH, sea level	-186.9	-1258.0
Cold day -12 C, sea level	-198.7	-1442.0
ISA at 1,600 m	-180.0	-1165.4

Table 2. Drop versus line of sight under different air conditions.

Choosing G1 versus G7

Figure 4 compares the same bullet described by a G1 BC of 0.462 and a G7 BC of 0.235 (chosen to be roughly equivalent at supersonic speed). The two models disagree most in the transonic zone because a real bullet's effective G1 BC shifts with speed while its G7 BC stays steadier. That is why G7, or a measured custom curve, is preferred for streamlined bullets at long range.

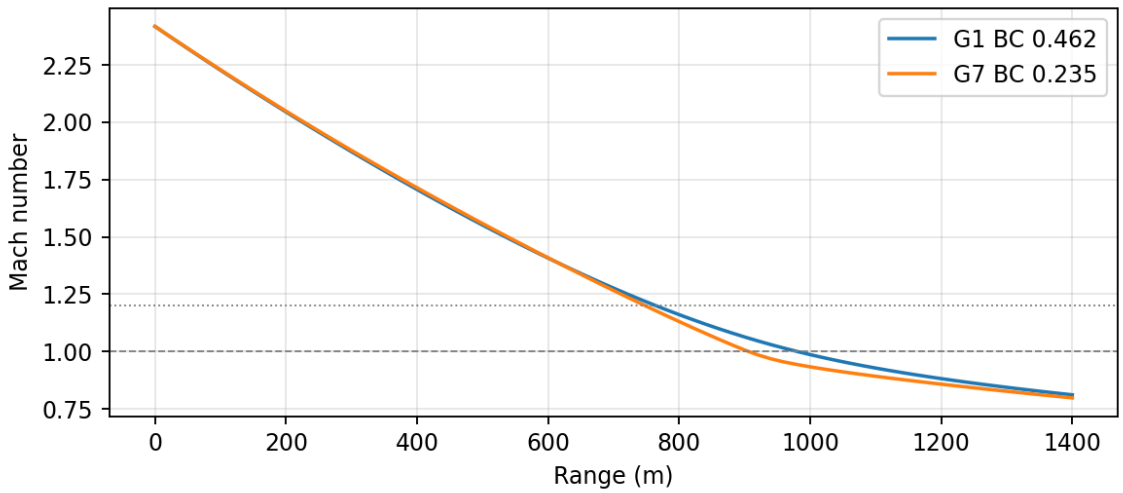


Figure 4. Mach number versus range for equivalent-looking G1 and G7 descriptions of one bullet.

7. Secondary effects in the example

For the same 1000 m shot the program isolates the smaller effects:

Effect	Magnitude	Comment
Spin drift (right twist)	+35.5 cm (right)	Depends on stability and TOF$^{1.83}$; not a wind effect.
Coriolis, horizontal, firing north at 40 deg N	+7.9 cm (right)	Sign reverses in the southern hemisphere.
Coriolis, vertical, firing east at 40 deg N	+9.3 cm (up)	Eotvos effect: eastward fire hits slightly high, westward slightly low.

Table 3. Magnitudes computed at 1000 m by the program.

Inclined fire

Because the bore angle was fixed at the flat zero, the drop below the line of sight at 800 m shrinks as the shot steepens:

Shot angle (deg)	0	15	30	45
Drop at 800 m (cm)	-696.9	-674.6	-599.5	-475.9

Table 4. Drop versus shot angle (same rifle, same zero). Steeper shots need less come-up than a flat table suggests.

How to use the output

A range table is only as good as its inputs: chronograph-measured velocity, a BC verified by field drop data, an accurate sight height and correct atmospheric data. The professional workflow is to enter the best available data, then truth the calculation at a long range by adjusting velocity or BC until the table matches observed impacts.

8. Verification and validation

8.1 Analytic vacuum test

Setting air density to zero removes drag and Coriolis, so the path must be the exact parabola $y = x \tan(a) - g x^2 / (2 v^2 \cos^2(a))$. A 100 m/s launch at 30 degrees:

x (m)	Simulated y (m)	Exact y (m)	Difference (m)
100	51.197260	51.197260	2.9e-07
200	89.318987	89.318987	1.9e-07
400	126.335841	126.335841	2.9e-07

x (m)	Simulated y (m)	Exact y (m)	Difference (m)
600	111.050561	111.050562	3.0e-07

Table 5. Vacuum trajectory versus exact solution.

8.2 Time-step convergence

dt (ms)	1000 m drop (cm)	Change from finest (cm)
4.000	-1332.671	0.0195
2.000	-1332.686	0.0046
1.000	-1332.689	0.0013
0.500	-1332.690	0.0002
0.250	-1332.690	0.0001
0.125	-1332.690	0.0000

Table 6. Convergence of the worked example. The default 0.5 ms step is converged to well under a millimetre at 1000 m, negligible next to the uncertainty in the inputs.

8.3 Built-in self-test

python ballistics.py --selftest runs eight checks: the vacuum parabola, time-step convergence, zero crossing at the zero range, monotonically decreasing speed, monotonic response to BC, wind sign convention, density falling with altitude, and the ICAO sea-level density of 1.225 kg/m^3. All pass in the delivered version.

8.4 What has and has not been validated

The integrator and the geometry (zeroing, frames, wind, Coriolis, incline) are verified against analytic or self-consistency tests. Physical fidelity for any given projectile depends on the drag data and the point-mass approximation. The program has not been benchmarked against Doppler-radar measurements, so anyone needing certified accuracy should compare against measured impacts or an established solver.

9. Assumptions and limitations

Assumption	Consequence
Point mass (3 degrees of freedom)	No yaw of repose, aerodynamic jump, Magnus force or lift; spin drift is an empirical add-on. Very accurate for flat-fire use, less so at extreme angles.
Constant air density and sound speed	Fine below about 1500 m of vertical change; ignores density loss on steep, long flights.
Uniform wind	Real wind varies with distance and height. Run several simulations with different winds to bracket the answer.
Reference drag curves	A single BC hides speed-dependent form-factor variation, especially with G1 near Mach 1. Use G7 or a custom curve where possible.
Constant Cd for balls	Spheres have a drag crisis at certain Reynolds numbers and spin adds lift; the presets are rough.
Spin-drift model	The Litz fit is valid for conventional stable bullets and is skipped when S_g is below 1.0.
Rigid Earth-fixed frame	Centrifugal terms are absorbed into g; adequate for the ranges considered.
No terminal ballistics or launcher modelling	By design; the program only predicts flight.

Safety note. A calculator does not replace safe practice. Always know your target, what is beyond it and the maximum range of the projectile, and follow local law and range rules.

10. Extending the program

Custom drag from a radar file: export Mach/Cd pairs to CSV and pass --drag-csv. **Variable wind:** replace the constant (wx, wy, wz) in fly() with a function of x; the small acc() function makes this a short change. **Density with altitude:** update rho inside the integrator from an ISA lapse model using the current height. **Batch or truing tools:** loop compute() over velocity or BC to fit measured drops. **GUI or web front end:** import the module and call compute(); every input is a plain dataclass.

11. Glossary

Term	Plain meaning
Ballistic coefficient (BC)	How well a projectile cuts through air compared with a standard shape. Higher means less slowing.
G1 / G7	Two standard reference shapes. Use the one your bullet's published BC refers to; do not mix them.
Drag coefficient (Cd)	How draggy a shape is. Streamlined shapes have low Cd; blunt shapes have high Cd.
Mach number	Speed divided by the speed of sound. Mach 1 is about 340 m/s on a standard day.
Transonic	The band around Mach 1 (this program flags below Mach 1.2) where drag changes quickly and accuracy suffers.
Line of sight	The straight line from the scope through the target. Drop is measured against it.
Zero	The distance at which the bullet path crosses the line of sight.
Sight height	How far the scope's centre sits above the barrel.
MOA / MIL	Angular units for scope adjustments: about 2.9 cm or 10 cm per 100 m respectively.
Time of flight (TOF)	Seconds between launch and arriving at a given range.
Windage	Sideways correction for wind (and other sideways effects).
Coriolis effect	A tiny sideways or vertical push caused by the Earth rotating during flight.
Spin drift	A slow sideways drift in the direction the bullet spins.
Sg (stability factor)	How steadily a spinning bullet keeps pointing forward. Roughly 1.5 or more is comfortable.
Station pressure	The actual air pressure where you stand, not the sea-level-adjusted value from weather reports.
Time step (dt)	How much time passes in each small step of the simulation.
Runge-Kutta 4 (RK4)	A standard, accurate method for stepping a simulation forward.
Dataclass	A Python labelled box that holds several related values.

12. Troubleshooting and FAQ

Question	Answer
'that zero range can't be reached'	The projectile is too slow, or the zero too far, to reach the line of sight within about 20 degrees of elevation. Check speed, units and zero distance.
Spin drift shows 0.	It needs --length and --twist, and a stability factor of at least 1.0. Check the Stability line in the report.
Plot was not created.	matplotlib is optional. Install it with pip install matplotlib and try again.
My numbers differ from another calculator.	Common causes: a different drag table (G1, G7 or custom), BC, sight height, weather or velocity. Enter identical inputs everywhere and compare a single range first.
The ball tables show huge MOA numbers.	For balls the scope-correction columns are not meaningful; read Drop as height above the launch point.
'need at least --mv, --weight and --diameter'	Those three are required unless you use --preset or run with no options and answer the questions.
Is the wind number 'from' or 'to'?	From. 90 means the wind blows from your right toward your left.
Which unit is the BC in?	Always lb/in2, as printed on ammunition data sheets, even though everything else is metric.
Is it fast enough?	Yes. A 1000 m table takes a fraction of a second. Raise --dt only for thousands of runs.
Can I trust it for competition or research?	It is verified against exact physics and self-consistency tests but not benchmarked against radar for particular projectiles. Confirm with real impact data and true the velocity or BC.

13. References

1. McCoy, R. L. Modern Exterior Ballistics: The Launch and Flight Dynamics of Symmetric Projectiles. Schiffer, 1999.

2. Litz, B. Applied Ballistics for Long Range Shooting. Applied Ballistics LLC (spin drift approximation and BC practice).

3. Miller, D. G. 'A New Rule for Estimating Rifling Twist.' Precision Shooting, 2005.

4. Buck, A. L. 'New equations for computing vapor pressure and enhancement factor.' J. Applied Meteorology 20, 1981.

5. ICAO. Manual of the ICAO Standard Atmosphere, Doc 7488.

6. Ingalls, J. M. Exterior Ballistics in the Plane of Fire (origin of the G-function drag models), 1917 edition.